

RETIRED 2026-08-12. Do not extend this draft. Its central mechanism, that search compute converts into quality only if the selection signal is refreshed, was falsified three ways: the same fixed replicate with four times the mutation rate beats the rotation arm; both deterministic yardsticks plateau while both time-varying ones keep improving, including one carrying no information; and the fixed arm's own believed score is flat, so it never learned its sample, it stopped searching. The search underneath also loses to matched-budget random search. See `RETIRED.md`; the measurement work continued in `~/bpnn`.

Teaching to the Test: How Much of an Architecture Search’s Improvement Is Not Real

Vasili Gavrilov*

August 2026

Abstract

Give a class the same practice exam every week and the marks will climb while the learning stops. An evolutionary architecture search does something very similar, for much the same reason, and this paper measures what it costs.

The search we were running would not do what it was asked. An earlier paper in this line had shown that the compact filter of a convolution appears on its own once mutation acts on a shared feature rather than on a single connection; the other half of a convolution, one filter reused at every position, it obtained by construction and said so. We built a task where that reuse genuinely pays, gave the search a genome free to refuse it, and waited. The search refused. A larger set of operators did not help, and neither did twelve times the generation budget. We assumed for some time that the fault was in the search, and it was not. The fault was in the number the search was being judged by.

A search scores each design on data held back from training, and because training is random a design can score well by luck. Keeping the best score keeps the luck. The remedy costs one line of bookkeeping: hold back two samples instead of one and choose using only the first. If the samples are interchangeable, then a design chosen without consulting either must score the same on both, so any gap between the sample used for choosing and the sample kept in reserve was manufactured by the choosing. That gap has an honest value of zero, needs no baseline, and is denominated in the same percentage points as the result being reported.

On NAS-Bench-101, a public table of 423,624 networks each trained three separate times, an ordinary-sized search overstates its accuracy by about half a percentage point, which is more than the improvements such searches are usually credited with. The consequence for compute is blunter than the consequence for honesty. Judged against one fixed held-out sample, our search reaches 94.18 after fifty generations and 94.19 after sixteen hundred, so a thirty-twofold increase in effort buys two thousandths of a point. Alternate between two samples, one per generation, and the same compute buys two tenths. The search was never short of compute. It was spending it to learn one sample more thoroughly.

Two repairs correct two different faults. Averaging two samples lowers the luck in every score and helps equally at any length of search. Alternating removes the fixed target instead, so its benefit compounds, and past roughly two hundred generations it overtakes averaging. On the small task, where the best design is fixed in advance and verified by enumeration, repairing the yardstick lifts exact recovery of that design from 16 runs in 240 to 42. The last finding is the one we like least and believe most: once the yardstick was fixed, the search began beating the design we had planted for it to find.

1 Where this continues from

The predecessor to this work [1] treated an evolutionary search as an instrument rather than an optimiser. Starting from a fully connected network and charging a price for every connection, it

*Independent researcher. Reference implementation: <https://github.com/Anode1/SMBPANN>.

asked which parts of a convolution the price alone would recover. Sparse task-relevant connectivity appeared readily, and weight sharing was adopted whenever the task rewarded recognising the same feature in more than one place. The compact local filter turned out to depend on a detail of the mutation operator, and that dependence was the paper’s contribution: mutation acting on one connection at a time cannot find the filter, while mutation acting on a shared offset can, because only the second form exposes the fact that sharing lets connection count and stored-weight count move independently.

What that paper did not obtain by search was the tiling, meaning the reuse of a single filter at every position. Its genome encoded connectivity as an offset mask, which makes every candidate translation invariant by construction, so the reuse was imposed rather than discovered. The paper said as much. Closing that gap is the obvious next step, and the obvious way to attempt it is to give the search a genome in which reuse is optional and a task in which reuse pays, then see whether the search elects it.

We built that task and ran that search, and for a long time the answer looked like a clean negative. The search settled on scattered supports rather than the compact aligned one, and neither a larger set of mutation operators nor twelve times the generation budget changed the outcome. A negative of that shape invites a structural explanation, and we spent a considerable amount of effort on structural explanations. The eventual cause was elsewhere, and it was not a property of the search at all. The quantity the search was selecting on did not measure what we were reporting, and once that was repaired the behaviour changed. Because the fault lies in the measurement rather than in the algorithm, it is not specific to convolutions, to evolutionary search, or to our task, which is why the bulk of this paper is about a public benchmark rather than about our own construction.

2 The problem, and the one change that measures it

Architecture search in its plainest form is trial and error with a scoreboard. A design is generated, trained, and scored on data that the training never saw. Good scores survive, are modified slightly, and are scored again. Nothing about this is controversial, and it works.

The difficulty enters through the randomness of training. Starting weights are drawn at random and the order of examples is shuffled, so the same design trained twice yields two different scores. Any single score is therefore the design’s real quality plus a contribution from luck, and selecting the largest score selects for both. The luck does not average away afterwards, because the design was chosen partly because of it. The reported number is consequently too high, and how much too high is not something a practitioner can read off the procedure.

An everyday version of the same arithmetic shows what is going on. Give a hundred students one examination and crown the highest scorer. That student is genuinely able, and also probably had a good day. Set a second examination and the score falls, not because the student has deteriorated, but because part of what the first examination selected was the good day, and good days do not repeat on demand. A student chosen at random rather than by score shows no such fall. The difference between the chosen and the unchosen is the entire quantity this paper measures.

2.1 Two held-out samples instead of one

Hold back two samples rather than one, choose using only the first, and leave the second untouched until the end.

Suppose the two samples are interchangeable, by which we mean they are the same kind of data separated at random, so that neither has any claim to be the more truthful. For a design chosen without consulting either of them, the expected score on the first equals the expected score on the second. That is not an assumption about the data or the model; it follows from the

way the split was made. Now take the design the search actually returned, which was chosen using the first sample, and compare its two scores. Any gap between them was produced by the act of choosing, because interchangeability has already excluded every other source.

We call that gap the inflation. It is worth stating three of its properties, because together they are what make it usable rather than merely correct. Its honest value is exactly zero, so it requires no control group and no baseline to be interpreted. It is expressed in the same percentage points of accuracy as the headline result, so it can be compared directly against whatever improvement is being claimed. And it costs one extra held-out sample, which in many settings is already available and simply being averaged away.

2.2 What is new, and what is not

Almost none of the statistics here is new, and it is worth naming the prior art before claiming anything at all. Selecting on a noisy measurement and then reporting that same measurement is a known bias, and the special case of taking a maximum over noisy candidates is the winner’s curse. That architecture searches overfit their validation signal is likewise known within the field, where the usual advice is to enlarge the validation set, resample it, or search less aggressively. **[PENDING: cite the specific NAS validation-overfitting literature, which has not yet been read and so is not yet claimed.]**

The additions are four. First, a measurement whose null value is zero by construction, which removes the usual difficulty of deciding what an overfitting number should be compared against. Second, the size of that number on real architecture data across four orders of magnitude of search effort, including the observation that it exceeds the accuracy differences these papers routinely treat as decisive. Third, evidence that two natural repairs address two different faults, one of which compounds with search length while the other does not, so that the better repair depends on how long the search runs and the two exchange places at a measurable point. Fourth, a controlled check on a task whose best design is fixed in advance and verified by enumeration, where repairing the yardstick roughly triples how often the search returns that design.

3 How large is it? Evidence from NAS-Bench-101

NAS-Bench-101 [2] is a published table of 423,624 network designs that have already been trained and scored, so searching over it costs a lookup instead of a training run. The property that matters here is incidental to its original purpose: every design was trained three separate times. Those three scores differ only through the randomness of training, which is exactly the condition for interchangeability, so the benchmark supplies the two held-out samples at no cost.

One practical detail nearly cost us the experiment, and it illustrates how easily this structure is discarded. The copy of the table in our own repository averages the three runs into a single number, which is the correct table for ranking designs and the wrong one for studying the yardstick, since averaging destroys precisely the structure the measurement depends on. We re-extracted from the original archive, keeping every run. As a check on the extraction, the mean of our three recovered scores reproduces the averaged table to within 10^{-4} percentage points on all 423,624 rows.

It is worth knowing how noisy one training run is before interpreting anything else. Across the benchmark, retraining the same design gives scores whose spread has a median of 0.33 percentage points, which sounds mild enough to ignore. The tail is not mild. At the 99th percentile the spread is 42.7 points, because roughly one design in a hundred sometimes trains successfully and sometimes collapses to the accuracy of guessing. A search that trusts a single training run will occasionally be shown the successful twin of a design that usually fails.

3.1 The measurement

Draw K designs at random, let the search keep whichever scores best on the first held-out sample, then look up that design’s score on the sample it was never shown. Here K stands for search effort. Alongside the inflation we record two quantities that matter more to somebody actually running a search: the true quality of the chosen design, taken as the mean of all three of its runs, and the regret, meaning how far short of the best design that was genuinely present in those K the search finished.

K	one held-out sample		two, averaged		inflation avoided
	inflation	regret	inflation	regret	
64	0.291	0.091	0.178	0.030	0.113
1,024	0.507	0.163	0.342	0.060	0.165
32,768	0.636	0.212	0.418	0.074	0.218
262,144	0.545	0.205	0.382	0.083	0.162

Every figure is in percentage points of accuracy over 100,000 repetitions per row, and the standard error on the final column is 0.0015 or below, so none of these differences is in question.

The inflation is large in the terms the field itself uses. At $K = 1024$, a search size nobody would describe as excessive, half a percentage point of the reported accuracy is not real, and papers in this area regularly contest differences smaller than that. The inflation also grows with effort, rising steadily from $K = 4$ to about $K = 32,768$, so the harder the search looks the greater the fraction of what it finds that is luck. Past that point it eases off slightly, for a reason we prefer to state rather than leave as an anomaly: once the sample approaches the size of the whole table the genuinely best designs are nearly always included, so the winner is increasingly a real winner. Averaging two samples helps on both counts at once, removing about a third of the inflation and cutting the regret by a factor between two and three. The second of those is the more important, because it means the repair changes which design is taken home and not merely what is said about it.

4 The same practice exam every week

Random search has no history, so it can only show what averaging buys. A generational search has history, and history is where the interesting fault lives.

Consider what a fixed held-out sample becomes over fifty generations. A design that happens to suit that particular sample keeps its unearned advantage, is copied into the next generation, and is copied again, and its descendants inherit whatever quirk of that sample favoured it. The sample stops being a test and becomes a syllabus. This is the practice exam given every week: the marks improve, and what improves them is familiarity with the paper rather than command of the subject. Averaging two samples does not address this at all, since it produces a better test and then sets the same better test forever. What addresses it is changing the paper, which costs nothing, because a second sample is already being held.

So the two repairs correct different faults, and the difference is mechanical rather than a matter of degree. Averaging reduces the luck present in any single score, two runs having $\sqrt{2}$ less spread than one, and that benefit arrives immediately and does not grow. Alternating leaves the luck in each score untouched, every generation still judging on one noisy run, and instead denies the population a fixed target to converge on. Accumulation takes time, so alternation should be worth nothing in a short search and progressively more as the search lengthens. We registered that prediction with its direction before running the sweep.

It holds, and the magnitude at long search lengths was larger than we expected. Below is a search with a population of 24 over the NAS-Bench-101 cell space, 20,000 runs per cell, every figure the true quality of the design returned, in percentage points.

generations	one fixed sample	alternating	averaging	alternating – fixed
10	94.139	94.132	94.238	−0.006
20	94.173	94.177	94.290	+0.004
50	94.183	94.230	94.313	+0.048
100	94.184	94.269	94.318	+0.085
200	94.185	94.318	94.314	+0.133
400	94.188	94.356	94.319	+0.169
800	94.187	94.395	94.316	+0.208
1600	94.185	94.430	94.315	+0.245

The second column is the result we consider the most useful thing in this paper, and it is not about accounting. A search judged throughout by one held-out sample arrives at 94.183 by fifty generations and at 94.185 by sixteen hundred. Thirty-two times the compute, two thousandths of a point. The same search with the paper changed each generation improves across the whole range and finishes a quarter of a point ahead. Search effort converts into better designs only while the yardstick is being refreshed; spent against a fixed one it goes into learning that sample more exactly, which the sample rewards and reality does not.

The final column shows the accumulation directly, worth nothing at ten generations and rising steadily thereafter. Setting the third and fourth columns side by side shows the two repairs changing places at around two hundred generations. That crossing is not a tuning artefact, since one fault is fixed in magnitude and the other compounds, so which repair dominates is decided by how long the search runs rather than by any parameter we chose.

[PENDING: the 3200-generation cell, running, to confirm the divergence continues.]

5 A controlled check, where the answer is known in advance

The benchmark establishes the size of the problem but cannot say whether repairing the yardstick lets a search find a particular structure, since nobody knows what NAS-Bench-101’s best design ought to look like. Answering that requires a task whose answer we fixed ourselves, which returns us to the question the first paper left open.

The task is a short strip of numbers containing a small pattern at one of ten positions, and the structure being sought is a convolution: one small filter, reused unchanged at every position. Two properties make the convolution genuinely the best answer rather than merely a plausible one. Training uses only three of the ten positions while scoring uses the remainder, so a design that cannot transfer to a position it never saw earns nothing, which is what makes reuse worth having. And each stored weight carries a price, so a filter wider than the pattern is penalised for the width it does not need. Reuse itself is a gene the search may switch off, which is the property the predecessor’s offset-mask genome lacked.

We verify that the target is the best answer rather than assuming it. Every one of the 127 filter shapes in the neighbourhood of the planted one is scored under the identical objective, and the planted shape ranks first. Two features of that check earned their inclusion through failure. It has to include shapes that are scattered rather than solid, because scattered shapes are what the search actually explores, and an enumeration restricted to solid ones would certify the target against the wrong family. And a rank is a statistic rather than a fact, since an argmax over noisy scores favours whichever candidate was luckiest, so we report the winning margin beside the standard error of the scores that produced it and believe the ordering only when the margin clears it. Applying that rule, the planted shape is the clear winner only when the price per weight falls within a band. Set the price too low and a wider filter genuinely wins, in which case a search that declines to return our target is behaving correctly and we would have been wrong to record a failure.

With the price inside that band we compare a search using one fixed held-out sample against a search using both repairs, over 240 runs matched on the same random draws, and score exact recovery, meaning that the design returned is the planted convolution itself.

	exact recovery, of 240 runs
one fixed held-out sample	16 (6.7%)
both repairs	42 (17.5%)
a random design of the same size	1 (0.4%)

The increase is roughly threefold, at $p = 7 \times 10^{-5}$ by an exact McNemar test on the paired runs. The final row is the control that carries the most weight, since a design with the same number of weights placed at random essentially never coincides with the target, which rules out the possibility that recovery is an accident of size.

5.1 What this result is not

The natural reading of the table above is more generous than the evidence supports, so we set out the limits explicitly.

It is not a claim that the search found the optimum. At the same setting the repaired search outperforms our planted convolution overall, by 0.031 on the objective at $p = 7 \times 10^{-8}$, using fewer weights at indistinguishable transfer accuracy. There is no contradiction between that and the enumeration, and the reconciliation is instructive: enumeration identifies the best single design averaged over many task draws, whereas the search chooses a design for the particular draw in front of it, and a choice made per task should beat the best fixed choice. Of the 198 runs that do not return the planted design, 136 score better than it on their own draw.

The figure of 17.5 per cent therefore needs something other than 100 per cent to be compared against. The planted design is the best answer for an individual draw about 13 per cent of the time, and that figure is itself a lower bound, since picking an argmax among 127 noisy candidates loses the true winner to lucky rivals. Set beside it, the repairs move recovery from roughly half the rate the task warrants to slightly above it, which is a smaller claim than saying the convolution emerges and is the claim the measurements will carry.

The controlled part of this evidence is also small by construction: one strip, ten positions, a single hidden layer. The mechanism concerns yardsticks rather than convolutions, which is why Section 3 exists and why we would not rest the general claim on this section alone.

6 Related work, and what it is owed

The statistical phenomenon is old. Selecting on a noisy measurement and then reporting that measurement is a known bias, and the maximum-over-candidates case is the winner’s curse. We claim novelty neither for the phenomenon nor for the remedies, but for the null of exactly zero, the magnitude on real architecture data, and the separation of the two faults.

Weight sharing as something learned rather than imposed has a substantial history: tying parameters through a prior [3], and driving locally connected layers toward convolutions either by regularisation or by the statistics of the data [4, 5], with the reachability of convolutional solutions under gradient descent studied directly [6]. The price per stored weight used here belongs to that family and is not new. **[PENDING: Nowlan and Hinton 1992, Elsayed et al. 2020, and d’Ascoli et al. 2019 must be read before these sentences stand.]**

Equivariance theory supplies the reason a convolution is the right structure to plant, since a linear map that commutes with a group action is a generalised convolution [7, 8]. The theorem concerns linear maps and exact symmetry, so it provides the target rather than a proof about our search space.

The lineage of the structure itself predates its use as an architecture. Local receptive fields repeated across positions were a finding about cortex [9] before the first architecture built them in using explicit position-invariant feature reuse [10], which preceded the weight-sharing formulation [11] that convolutional networks inherit. **[PENDING: confirm Hubel and Wiesel 1962 and Fukushima 1980 have been read.]**

7 Conclusion

A search that judges candidates on held-out data is trusting a number that contains luck, and taking the best of many such numbers retains the luck. Holding out two interchangeable samples rather than one makes the retained luck visible as a gap whose honest value is zero. On real architecture data that gap is about half a percentage point at ordinary search sizes, which is larger than the improvements the field argues over, and it grows as the search works harder.

Two repairs cost almost nothing and correct different faults. Averaging two samples reduces the luck in each score and helps equally at any search length. Alternating between them removes the fixed target that a long search learns to please, so it helps more the longer the search runs, and past a few hundred generations it overtakes averaging. The sharpest consequence is about compute rather than about statistics: against a fixed yardstick a search stops improving after about fifty generations and a sixteenfold increase in effort adds nothing, while the same search with a refreshed yardstick keeps improving throughout.

The first paper in this line asked which pieces of a convolution a search recovers on its own and answered it for the filter. This one takes up the piece that was left imposed and finds that the governing constraint sits in the instrument rather than in the search, which is a less satisfying answer about convolutions and a more useful one about method. Before asking whether a search is good enough, it is worth checking that the number it is being judged by measures what will eventually be reported. In our case that check amounts to one additional column in a results table, and it retired two negative results we had already believed.

Appendix: how each result regenerates

Every probe is a single self-contained C99 file under `validation/paper2/` in the reference implementation, every number in this paper comes from an archived output file in the same repository, and environment variables select the mode. Defaults are preserved throughout, and after each change a binary built from the previous source is confirmed to reproduce the previous result table byte for byte.

result	invocation
per-run scores recovered from the archive	<code>./nb101_trials nasbench_only108.tfrecored</code>
inflation against search effort (§3)	<code>KMIN=2 KMAX=262144 REPS=100000 ./nb101_signal</code>
averaging against alternating (§4)	<code>POP=24 GENS=\$G RUNS=20000 ./nb101_search</code>
the planted-target enumeration (§5)	<code>SUBSCAN=1 SUBW=7 SEEDS=60 DRAWS=12</code>
exact recovery (§5)	<code>RAW=1 ROTPOS={0,1} FEVAL={1,12} LAMBDA=6</code>
paired statistics	<code>GROUP=2 METRICS=... ./pairstat</code>

Paired continuous outcomes are compared with the Wilcoxon signed-rank test as primary, using an exact null where ties permit and a tie-corrected normal approximation otherwise, always reporting which was used, together with Hodges-Lehmann estimates and distribution-free intervals, a paired t as secondary, Holm correction across the declared family, and a minimum detectable effect so that a null result is bounded rather than merely asserted. Paired binary

outcomes use an exact McNemar test, computed as a two-sided binomial on the discordant pairs. The tool implementing all of this is checked against hand-computable cases before use.

Predictions for each experiment were written to a timestamped file before the corresponding run, together with what would refute them, and scored afterwards. Two were wrong and are recorded as such: on the small task we predicted that neither repair would suffice alone, and averaging alone did more than alternating alone; and on the benchmark we predicted that alternating would prove inert at every search length, which held only for short searches.

References

- [1] V. Gavrilov. The Imposed and Emergent Pieces of Convolution Under an Energy Budget. Zenodo 10.5281/zenodo.21423177, 2026.
- [2] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, F. Hutter. NAS-Bench-101: Towards Reproducible Neural Architecture Search. *ICML* 2019.
- [3] S. J. Nowlan, G. E. Hinton. Simplifying Neural Networks by Soft Weight Sharing. *Neural Computation* 4(4), 1992.
- [4] B. Neyshabur. Towards Learning Convolutions from Scratch. *NeurIPS* 2020.
- [5] A. Ingrosso, S. Goldt. Data-driven emergence of convolutional structure in neural networks. *PNAS* 119(40), 2022.
- [6] S. d’Ascoli, L. Sagun, J. Bruna, G. Biroli. Finding the Needle in the Haystack with Convolutions. *NeurIPS* 2019.
- [7] R. Kondor, S. Trivedi. On the Generalization of Equivariance and Convolution in Neural Networks to the Action of Compact Groups. *ICML* 2018.
- [8] T. Cohen, M. Welling. Group Equivariant Convolutional Networks. *ICML* 2016.
- [9] D. H. Hubel, T. N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex. *J. Physiology* 160(1), 1962.
- [10] K. Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics* 36, 1980.
- [11] Y. LeCun et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.